МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ

«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

ФАКУЛЬТЕТ ІНФОРМАТИКИ ТА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

КАФЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

Лабораторна робота №3

з дисципліни «Алгоритми та структури даних»

Виконав:

студент групи ІМ-о51

Машута О.І

Перевірив:

ас. Череватенко О.В.

Київ 2026

Постановка задачі:

1. Представити у програмі напрямлений і ненапрямлений графи з задани ми параметрами: • кількість вершин n; • розміщення вершин; • матриця суміжності A

2. Створити програму для формування зображення напрямленого і нена прямленого графів у графічному вікні

Згадані вище параметри графа задаються на основі чотиризначного но мера варіанту n1n2n3n4, де n1n2 це десяткові цифри номера групи, а n3n4 — десяткові цифри номера варіанту, який був у студента для двох попередніх робіт (див. таблицю з поточними оцінками з АСД, надану викладачем на початку поточного семестру).

Кількість вершин n дорівнює 10 n3. Розміщення вершин: • колом при n4 0,1; • квадратом (прямокутником) при n4 2,3; • трикутником при n4 4,5; 38 • колом з вершиною в центрі при n4 6,7; • квадратом (прямокутником) з вершиною в центрі при n4 8,9

Матриця суміжності Adir напрямленого графа за варіантом фор мується таким чином

1) встановлюється параметр (seed) генератора випадкових чисел, рівний номеру варіанту n1n2n3n4 — детальніше див. с. 12

2) матриця розміром n n заповнюється згенерованими випадковими чи слами в діапазоні [0, 2.0)

3) обчислюється коефіцієнт k 1.0 n3 0.02 n4 0.005 0.25;

4) кожен елемент матриці множиться на коефіцієнт k

5) елементи матриці округлюються: 0 — якщо елемент менший за 1.0, 1 — якщо елемент більший або дорівнює 1.0

Матриця суміжності Aundir ненапрямленого графа одержується з ма триці Adir: adiri,j 1 ñ aundiri,j 1,aundirj,i 1. Наприклад, якщо запрограмувати додаткові функції randm та mulmr, у програмі на мові C генерація матриці Adir напрямленого графа може вигля дати так 1 srand(n1n2n3n4); 2 T = randm(n); 3 k = 1.0- n3*0.02- n4*0.005- 0.25; 4 A = mulmr(T, k);

Тут randm(n) — розроблена функція, яка формує матрицю розміром nn, що складається з випадкових чисел у діапазоні (0, 2.0); mulmr(T, k) — розроблена функція множення матриці на коефіцієнт та окру глення результату до 0 чи 1

При проєктуванні програм слід врахувати наступне

1) мова програмування обирається студентом самостійно

2) графічне зображення графа має формуватися на основі графічних при мітивів з графічної бібліотеки (таких як еліпс, пряма, дуга, текст тощо)

3) використання готових бібліотек для роботи з графами не дозволяється

4) вивід графа має бути реалізований універсальним чином: вершини і ребра мають виводитися в циклі, а не окремими командами для кожного графічного елемента

5) типи та структури даних для внутрішнього представлення графа у про грамі слід вибрати самостійно

6) матриці суміжності графів можна виводити в графічне вікно або кон соль — на розсуд студента

7) матриці суміжності мають виводитися як матриці: в квадратному ви гляді, з 1 та 0

Текст програми:

using System; using System.Collections.Generic; using System.Drawing; using System.Drawing.Drawing2D; using System.Linq; using System.Windows.Forms;

namespace GraphVisualization { public class Vertex { public int Id { get; set; } public float X { get; set; } public float Y { get; set; } public const int Radius = 15;

```
   public Vertex(int id, float x, float y)
    {
        Id = id;
        X = x;
        Y = y;
    }
}


public class Graph
{
    public int N { get; private set; }
    public bool IsDirected { get; set; }
    public int[,] AdjMatrix { get; set; }
    public List<Vertex> Vertices { get; set; }

    public Graph(int n, bool isDirected)
    {
        N = n;
```

```
        IsDirected = isDirected;
        Vertices = new List<Vertex>();
        AdjMatrix = new int[n, n];
    }
}

public class GraphPanel : Panel
{
    private Graph? _graph;

    public GraphPanel()
    {
        this.DoubleBuffered = true;
        this.BackColor = Color.White;
    }

    public void SetGraph(Graph graph)
    {
        _graph = graph;
        this.Invalidate();
    }

    protected override void OnPaint(PaintEventArgs e)
    {
        base.OnPaint(e);
        if (_graph == null || _graph.Vertices.Count == 0) return;

        Graphics g = e.Graphics;
        g.SmoothingMode = SmoothingMode.AntiAlias;

        using Pen edgePen = new Pen(Color.Black, 1.5f);

        if (_graph.IsDirected)
        {
            AdjustableArrowCap bigArrow = new AdjustableArrowCap(6,
6);
            edgePen.CustomEndCap = bigArrow;
        }

        using Font font = new Font("Arial", 10, FontStyle.Bold);
        Brush vertexBrush = Brushes.LightGray;
        Pen vertexBorder = Pens.Black;
```

```
        Brush textBrush = Brushes.Black;

        int r = Vertex.Radius;

        for (int i = 0; i < _graph.N; i++)
        {
            for (int j = 0; j < _graph.N; j++)
            {
                if (_graph.AdjMatrix[i, j] == 1)
                {
                    if (!_graph.IsDirected && i > j) continue;
                    if (i == j) continue;

                    PointF start = new PointF(_graph.Vertices[i].X,
_graph.Vertices[i].Y);
                    PointF end = new PointF(_graph.Vertices[j].X,
_graph.Vertices[j].Y);

                    AdjustLinePoints(ref start, ref end, r);

                    if (_graph.IsDirected && _graph.AdjMatrix[j, i]
== 1)
                    {
                        OffsetLine(ref start, ref end, 5);
                    }

                    g.DrawLine(edgePen, start, end);
                }
            }
        }

        foreach (var v in _graph.Vertices)
        {
            g.FillEllipse(vertexBrush, v.X - r, v.Y - r, 2 * r, 2 *
r);
            g.DrawEllipse(vertexBorder, v.X - r, v.Y - r, 2 * r, 2 *
r);

            SizeF textSize = g.MeasureString((v.Id + 1).ToString(),
font);
            g.DrawString((v.Id + 1).ToString(), font, textBrush,
                v.X - textSize.Width / 2, v.Y - textSize.Height /
```

```
2);
        }
    }

    private void AdjustLinePoints(ref PointF start, ref PointF end,
int radius)
    {
        float dx = end.X - start.X;
        float dy = end.Y - start.Y;
        float length = (float)Math.Sqrt(dx * dx + dy * dy);

        if (length == 0) return;

        float ratio = radius / length;

        start.X += dx * ratio;
        start.Y += dy * ratio;

        end.X -= dx * ratio;
        end.Y -= dy * ratio;
    }

    private void OffsetLine(ref PointF start, ref PointF end, float
offset)
    {
        float dx = end.X - start.X;
        float dy = end.Y - start.Y;
        float len = (float)Math.Sqrt(dx * dx + dy * dy);
        if (len == 0) return;

        float nx = -dy / len * offset;
        float ny = dx / len * offset;

        start.X += nx;
        start.Y += ny;
        end.X += nx;
        end.Y += ny;
    }
}

public static class GraphGenerator
{
```

```
    public static int[,] GenerateDirectedAdjacencyMatrix(int n, int
seed, int n3, int n4)
    {
        int[,] matrix = new int[n, n];
        Random rand = new Random(seed);

        double k = 1.0 - n3 * 0.02 - n4 * 0.005 - 0.25;

        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (i == j) { matrix[i, j] = 0; continue; }
                double val = rand.NextDouble() * 2.0;
                double scaled = val * k;
                matrix[i, j] = scaled >= 1.0 ? 1 : 0;
            }
        }
        return matrix;
    }

    public static int[,] GenerateUndirectedAdjacencyMatrix(int[,]
dirMatrix, int n)
    {
        int[,] matrix = new int[n, n];
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (dirMatrix[i, j] == 1 || dirMatrix[j, i] == 1)
                {
                    matrix[i, j] = 1;
                    matrix[j, i] = 1;
                }
            }
        }
        return matrix;
    }

    public static void PlaceVertices(List<Vertex> vertices, int n,
int n4, float centerX, float centerY, float width, float height)
    {
```

```
        vertices.Clear();
        float margin = 40f;
        float radiusX = Math.Max(20, Math.Min(width, height) / 2f -
margin);
        float radiusY = radiusX;

        int mode = n4 % 10;

        if (mode == 0 || mode == 1)
        {
            for (int i = 0; i < n; i++)
            {
                double angle = 2 * Math.PI * i / n;
                float x = centerX + (float)(radiusX *
Math.Cos(angle));
                float y = centerY + (float)(radiusY *
Math.Sin(angle));
                vertices.Add(new Vertex(i, x, y));
            }
            return;
        }

        if (mode == 2 || mode == 3)
        {
            PlaceAlongRectanglePerimeter(vertices, n, centerX,
centerY, radiusX * 1.4f, radiusY * 0.9f);
            return;
        }

        if (mode == 4 || mode == 5)
        {
            PlaceAlongTrianglePerimeter(vertices, n, centerX,
centerY, radiusX * 1.2f);
            return;
        }

        if (mode == 6 || mode == 7)
        {
            if (n >= 1)
            {
                vertices.Add(new Vertex(0, centerX, centerY));
                int m = n - 1;
```

```
                for (int i = 0; i < m; i++)
                {
                    double angle = 2 * Math.PI * i / m;
                    float x = centerX + (float)(radiusX *
Math.Cos(angle));
                    float y = centerY + (float)(radiusY *
Math.Sin(angle));
                    vertices.Add(new Vertex(i + 1, x, y));
                }
            }
            return;
        }

        if (n >= 1)
        {
            vertices.Add(new Vertex(0, centerX, centerY));
            int m = n - 1;
            PlaceAlongRectanglePerimeter(vertices, m, centerX,
centerY, radiusX * 1.4f, radiusY * 0.9f, startId:1);
        }
    }

    private static void PlaceAlongRectanglePerimeter(List<Vertex>
vertices, int n, float cx, float cy, float halfW, float halfH, int
startId = 0)
    {
        if (n <= 0) return;
        float left = cx - halfW;
        float right = cx + halfW;
        float top = cy - halfH;
        float bottom = cy + halfH;

        float perimeter = 2 * (halfW * 2 + halfH * 2);
        for (int i = 0; i < n; i++)
        {
            float t = (i / (float)n) * perimeter;
            float x, y;
            if (t < (halfW * 2))
            {
                x = left + t;
                y = top;
            }
```

```
            else if (t < (halfW * 2 + halfH * 2))
            {
                float tt = t - (halfW * 2);
                x = right;
                y = top + tt;
            }
            else if (t < (halfW * 4 + halfH * 2))
            {
                float tt = t - (halfW * 2 + halfH * 2);
                x = right - tt;
                y = bottom;
            }
            else
            {
                float tt = t - (halfW * 4 + halfH * 2);
                x = left;
                y = bottom - tt;
            }
            vertices.Add(new Vertex(startId + i, x, y));
        }
    }

    private static void PlaceAlongTrianglePerimeter(List<Vertex>
vertices, int n, float cx, float cy, float radius)
    {
        if (n <= 0) return;
        PointF p1 = new PointF(cx, cy - radius);
        PointF p2 = new PointF(cx - radius * (float)Math.Sin(Math.PI
/ 3), cy + radius * 0.5f);
        PointF p3 = new PointF(cx + radius * (float)Math.Sin(Math.PI
/ 3), cy + radius * 0.5f);

        float side1 = Distance(p1, p2);
        float side2 = Distance(p2, p3);
        float side3 = Distance(p3, p1);
        float perimeter = side1 + side2 + side3;

        for (int i = 0; i < n; i++)
        {
            float t = (i / (float)n) * perimeter;
            PointF pt;
            if (t < side1)
```

```
            {
                pt = InterpolateAlong(p1, p2, t / side1);
            }
            else if (t < side1 + side2)
            {
                pt = InterpolateAlong(p2, p3, (t - side1) / side2);
            }
            else
            {
                pt = InterpolateAlong(p3, p1, (t - side1 - side2) /
side3);
            }
            vertices.Add(new Vertex(i, pt.X, pt.Y));
        }
    }

    private static float Distance(PointF a, PointF b)
    {
        float dx = a.X - b.X;
        float dy = a.Y - b.Y;
        return (float)Math.Sqrt(dx * dx + dy * dy);
    }

    private static PointF InterpolateAlong(PointF a, PointF b, float
t)
    {
        return new PointF(a.X + (b.X - a.X) * t, a.Y + (b.Y - a.Y) *
t);
    }

    public static string MatrixToString(int[,] matrix, int n)
    {
        var lines = new List<string>();
        for (int i = 0; i < n; i++)
        {
            var row = new List<string>();
            for (int j = 0; j < n; j++)
                row.Add(matrix[i, j].ToString());
            lines.Add(string.Join(" ", row));
        }
        return string.Join(Environment.NewLine, lines);
    }
```

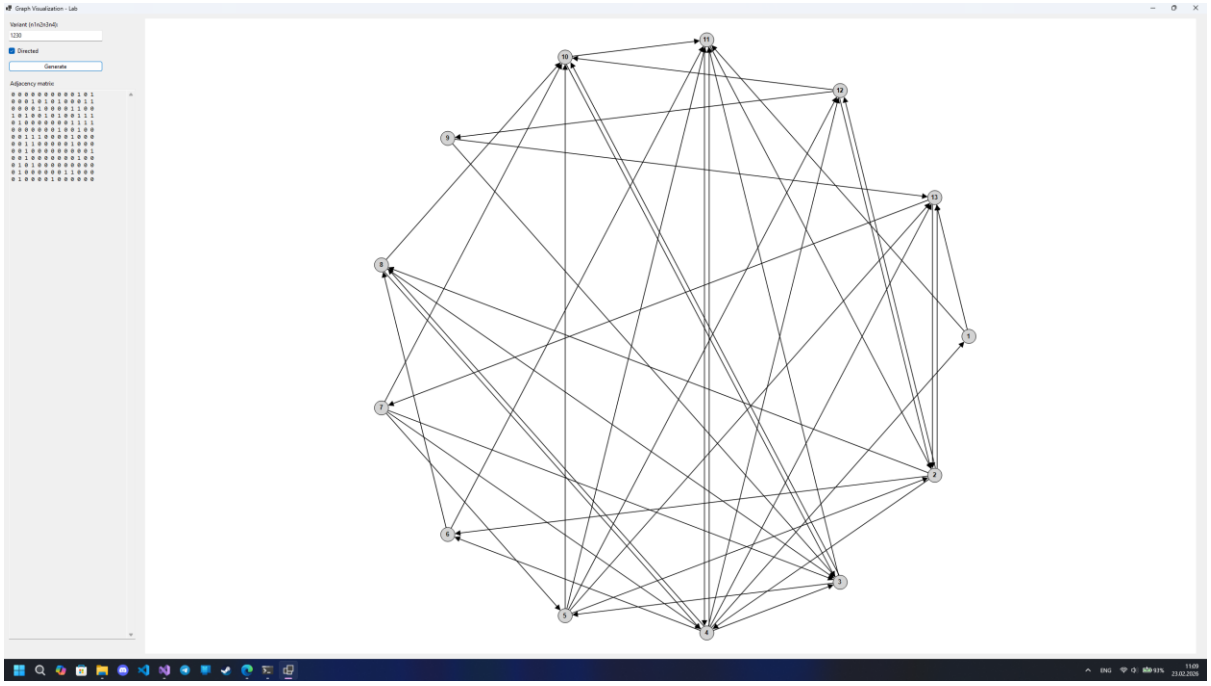
```
}

public static class Program
{
    [STAThread]
    public static void Main()
    {
        Application.SetHighDpiMode(HighDpiMode.SystemAware);
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);

        var form = new Form
        {
            Text = "Graph Visualization - Lab",
            Width = 1100,
            Height = 800
        };

        var panel = new GraphPanel
        {
            Location = new Point(300, 10),
            Size = new Size(760, 740),
            Anchor = AnchorStyles.Top | AnchorStyles.Bottom |
AnchorStyles.Left | AnchorStyles.Right
        };
        form.Controls.Add(panel);

        var inputLabel = new Label { Text = "Variant (n1n2n3n4):",
Location = new Point(10, 15), AutoSize = true };
        var variantBox = new TextBox { Location = new Point(10, 35),
Width = 200, Text = "1234" };

        var directedCheck = new CheckBox { Text = "Directed",
Location = new Point(10, 70), AutoSize = true, Checked = true };

        var generateBtn = new Button { Text = "Generate", Location =
new Point(10, 100), Width = 200 };

        var matrixLabel = new Label { Text = "Adjacency matrix:",
Location = new Point(10, 140), AutoSize = true };
        var matrixBox = new TextBox { Location = new Point(10, 160),
Size = new Size(270, 560), Multiline = true, ScrollBars =
```

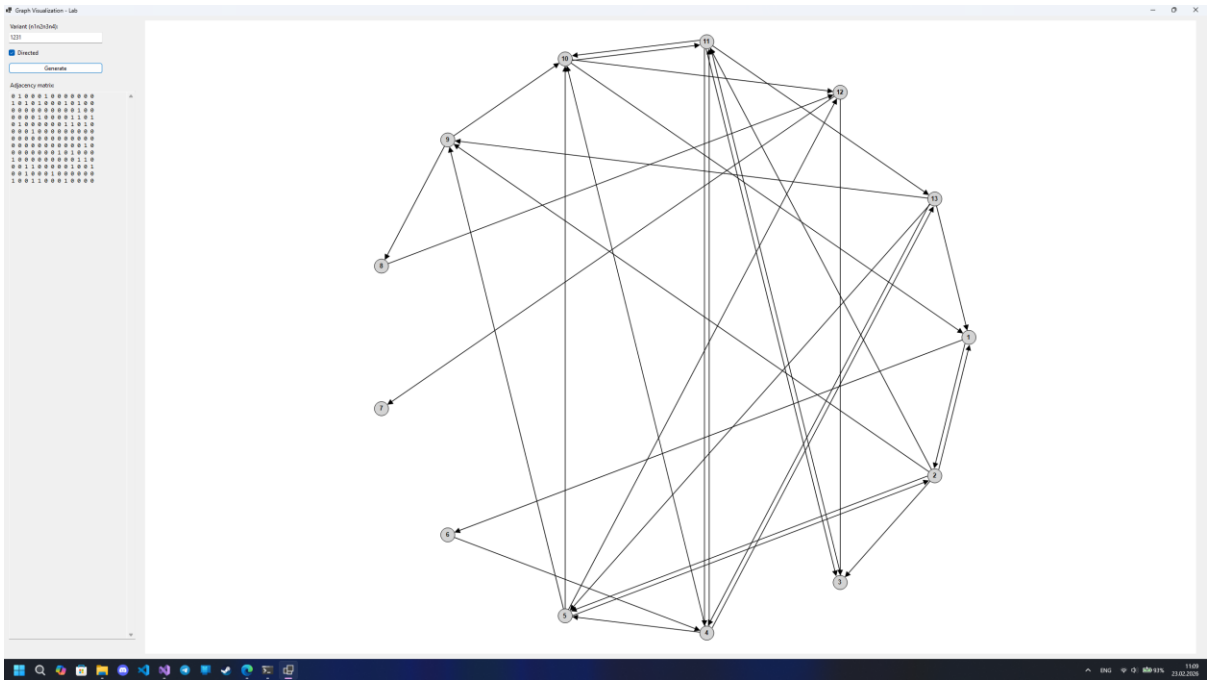
```
ScrollBars.Both, Font = new Font("Consolas", 10), ReadOnly = true,
Anchor = AnchorStyles.Top | AnchorStyles.Bottom | AnchorStyles.Left
};

        form.Controls.Add(inputLabel);
        form.Controls.Add(variantBox);
        form.Controls.Add(directedCheck);
        form.Controls.Add(generateBtn);
        form.Controls.Add(matrixLabel);
        form.Controls.Add(matrixBox);

        void DoGenerate()
        {
            string v = variantBox.Text.Trim();
            if (v.Length != 4 || !v.All(char.IsDigit))
            {
                MessageBox.Show("Enter 4 decimal digits
(n1n2n3n4).", "Invalid variant", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
                return;
            }

            int n1 = v[0] - '0';
            int n2 = v[1] - '0';
            int n3 = v[2] - '0';
            int n4 = v[3] - '0';

            int n = 10 + n3;

            if (n <= 0)
            {
                MessageBox.Show("n computed as number of vertices
must be > 0", "Invalid n", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
                return;
            }

            int seed = n1 * 1000 + n2 * 100 + n3 * 10 + n4;

            var graph = new Graph(n, directedCheck.Checked);
            int[,] dir =
GraphGenerator.GenerateDirectedAdjacencyMatrix(n, seed, n3, n4);
```

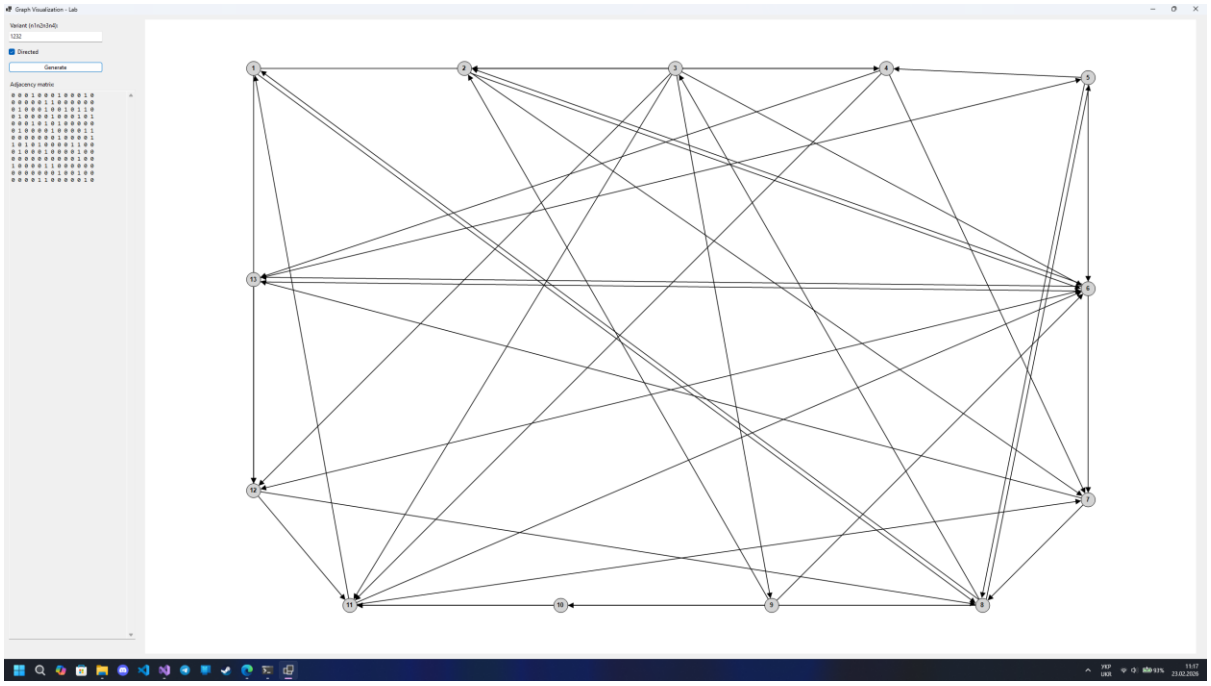
```
            if (directedCheck.Checked)
            {
                graph.AdjMatrix = dir;
                graph.IsDirected = true;
            }
            else
            {
                graph.AdjMatrix =
GraphGenerator.GenerateUndirectedAdjacencyMatrix(dir, n);
                graph.IsDirected = false;
            }

            float cx = panel.ClientSize.Width / 2f;
            float cy = panel.ClientSize.Height / 2f;
            GraphGenerator.PlaceVertices(graph.Vertices, n, n4, cx,
cy, panel.ClientSize.Width, panel.ClientSize.Height);

            panel.SetGraph(graph);
            matrixBox.Text =
GraphGenerator.MatrixToString(graph.AdjMatrix, n);
        }

        generateBtn.Click += (s, e) => DoGenerate();

        panel.Resize += (s, e) =>
        {
            if (panel.ClientSize.Width > 0 &&
panel.ClientSize.Height > 0 &&
!string.IsNullOrEmpty(matrixBox.Text))
            {
                DoGenerate();
            }
        };

        DoGenerate();

        Application.Run(form);
    }
}
```

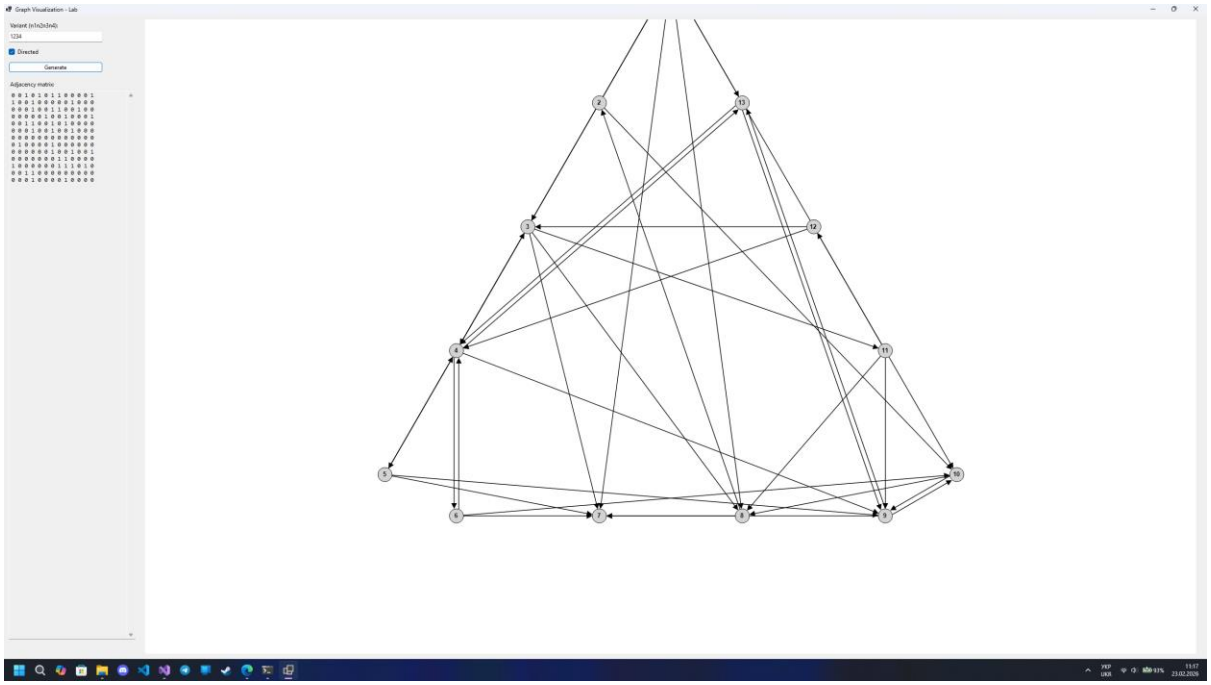
}

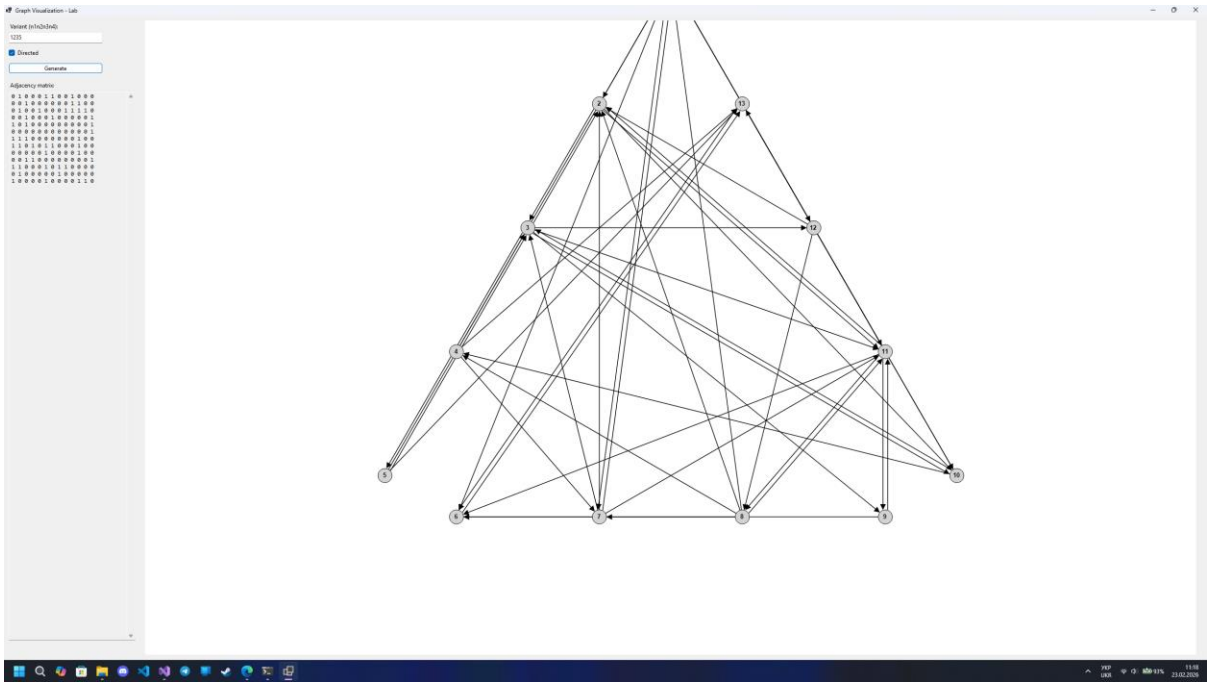
Тест програми:

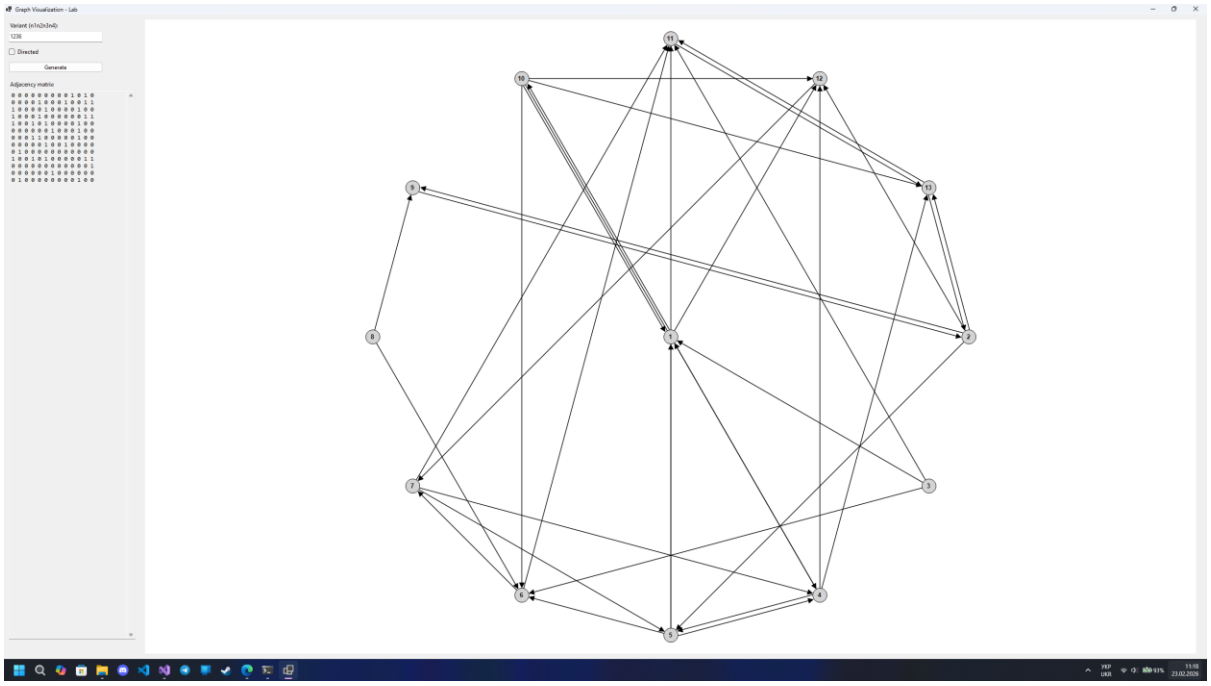


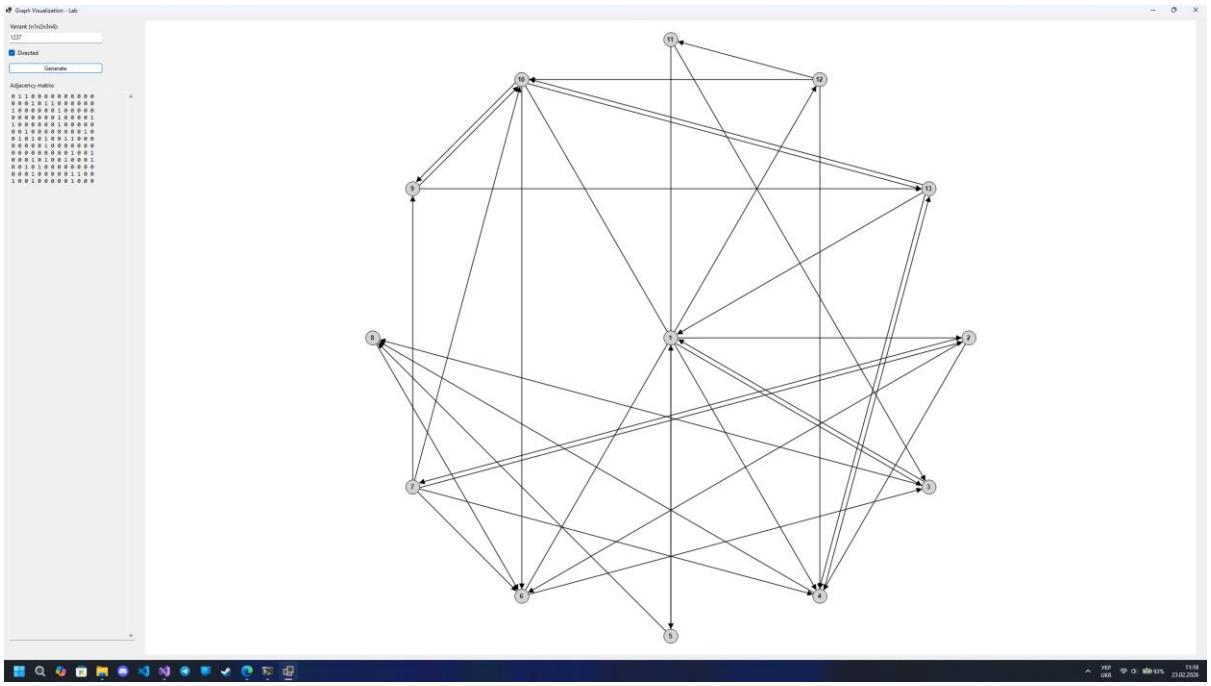


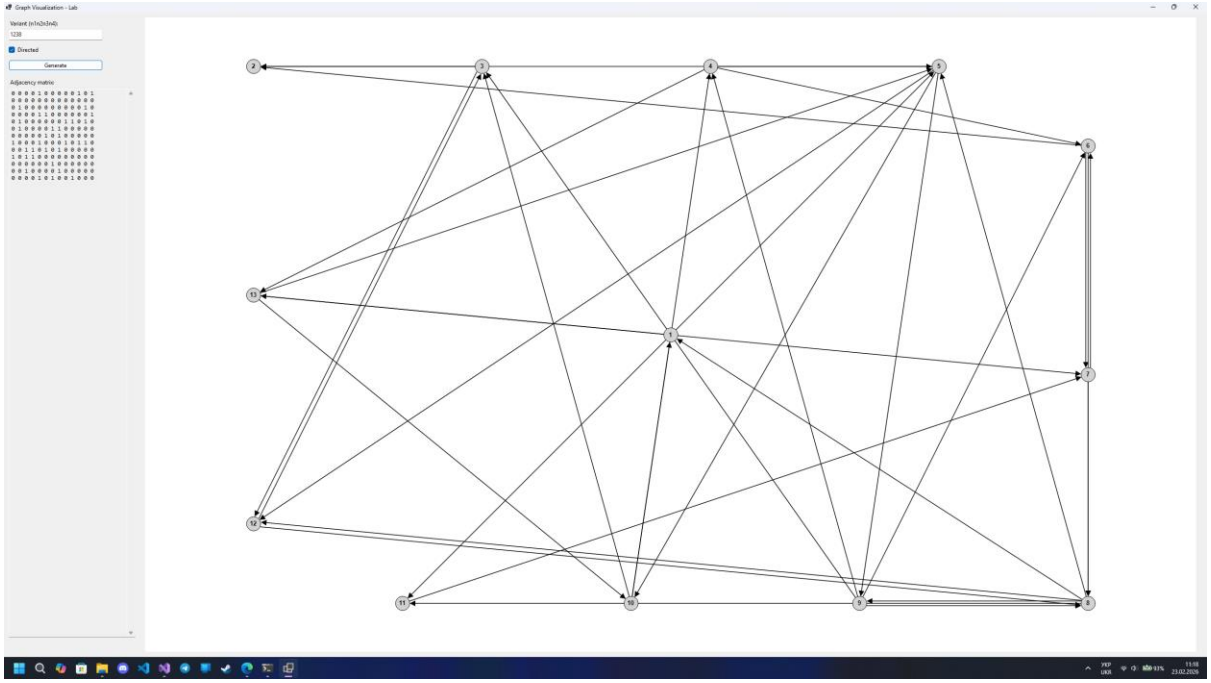

Graph Visualization - Lab
Variant (n1n2n3n4):
1232
Directed
Generate
Adjacency matrix
0 0 0 1 0 0 0 1 0 0 0 1 0
0 0 0 0 0 1 1 0 0 0 0 0 0
0 1 0 0 0 1 0 0 1 0 1 1 0
0 1 0 0 0 1 0 0 0 1 0 0 1
0 0 0 1 0 1 0 1 0 0 0 0 0
0 1 0 0 0 0 1 0 0 0 0 1 1
0 0 0 0 0 0 0 1 0 0 0 0 1
1 0 1 0 1 0 0 0 0 1 1 0 0
0 1 0 0 0 1 0 0 0 0 1 0 0
0 0 0 0 0 0 0 0 0 0 0 1 0
1 0 0 0 0 1 1 0 0 0 0 0 0
0 0 0 0 0 0 0 1 1 0 0 0 0
0 0 0 0 0 1 1 0 0 0 0 0 1
0 0 0 0 1 1 0 0 0 0 0 1 0
1
2
3
4
5
13
6
12
7
11
10
9
8
Graph Visualization - Lab
Variant (n1n2n3n4):
1233
Directed
Generate
Adjacency matrix
0 1 0 0 0 1 0 0 0 1 0 0 0
0 0 1 0 0 0 0 0 0 0 0 0 1
0 0 0 0 0 0 0 0 1 0 0 1 0
0 0 0 0 0 0 0 1 1 1 0 1 1
0 0 0 0 0 0 0 0 0 0 1 0 0
0 1 0 0 0 0 1 1 0 0 0 0 0
1 0 0 1 0 0 0 0 1 1 0 0 1
0 1 0 0 1 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 1 0 0 1 0 0
1 0 0 0 0 1 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0 0 1 0
1 0 0 0 0 0 0 1 0 0 0 0 0
1
2
3
4
5
13
6
12
7
11
10
9
8

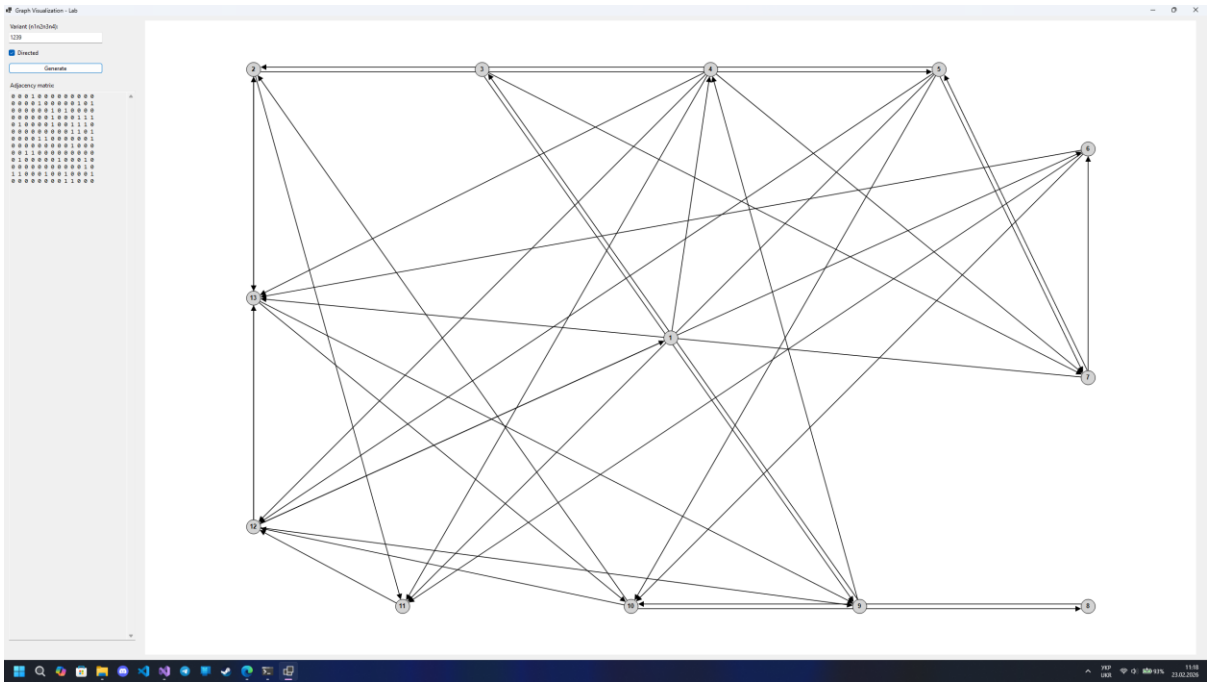

Graph Visualization - Lab
Variant (n1n2n3n4):
1234
Directed
Generate
Adjacency matrix
0 0 1 0 1 0 1 1 0 0 0 0 1
1 0 0 1 0 0 0 0 1 0 0 0
0 0 0 1 0 0 1 1 0 0 1 0 0
0 0 0 0 0 1 0 0 1 0 0 0 1
0 0 1 1 0 0 1 0 1 0 0 0
0 0 0 1 0 0 1 0 0 1 0 0
0 0 0 0 0 0 0 0 0 0 0 0
0 1 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 1 0 0 1 0 0 1
0 0 0 0 0 0 0 1 1 0 1 0
1 0 0 0 0 0 0 1 1 1 0 1 0
0 0 1 1 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 0 1 0 0 0 0
2
13
3
12
4
11
5
10
6
7
8
9


Graph Visualization - Lab
Variant (n1n2n3n4):
1235
Directed
Generate
Adjacency matrix
0 1 0 0 0 1 1 0 1 0 0 0
0 0 1 0 0 0 0 0 0 1 1 0
0 1 0 0 1 0 0 1 1 1 1 0
0 0 1 0 0 1 1 0 0 0 0 1
1 0 1 0 0 0 0 0 0 0 0 1
0 0 0 0 0 0 0 0 0 0 0 1
1 1 0 0 0 0 0 1 0 0 1 0
1 1 0 1 0 1 1 0 0 1 1 0
0 0 0 0 0 0 0 0 0 0 0 0
0 0 1 1 0 0 0 0 0 0 0 1
1 0 0 0 1 1 0 0 0 0 0 0
0 1 0 0 0 0 1 0 0 0 0 0
1 0 0 0 0 1 0 0 0 0 1 1 0
2
13
3
12
4
11
5
10
6
7
8
9


Graph Visualization - Lab
Variant (n1n2n3n4):
1238
Directed
Generate
Adjacency matrix
0 0 0 0 0 0 0 0 0 1 0 1 0
0 0 0 0 1 0 0 1 0 0 1 1
1 0 0 0 0 1 0 0 0 0 1 0 0
1 0 0 0 1 0 0 0 0 0 0 1 1
1 0 0 1 0 1 0 1 0 0 0 1 0 0
0 0 0 0 0 1 0 0 1 0 0 1 0
0 0 0 1 1 0 0 0 0 0 1 0 0
0 0 0 0 0 1 0 1 0 0 0 0 0
0 1 0 0 0 0 0 0 0 0 0 0 0
1 0 0 1 1 0 0 0 0 0 1 1 1
0 0 0 0 0 0 0 0 0 0 0 0 1
0 0 0 0 0 0 0 0 0 0 0 0 0
0 1 0 0 0 0 0 0 0 1 0 0 0
1
2
3
4
5
6
7
8
9
10
11
12


Graph Visualization - Lab
Variant (n1n2n3n4):
1227
Directed
Generate
Adjacency matrix
0 1 1 0 0 0 0 0 0 0 0 0
0 0 0 1 0 1 1 0 0 0 0 0
1 0 0 0 0 1 0 1 0 0 0 0
0 0 0 0 0 0 1 1 0 0 0 1
1 0 0 0 0 0 0 0 1 0 0 0
0 0 1 0 0 0 0 0 0 0 1 0
0 1 0 1 0 0 1 0 0 0 0 0
0 0 0 0 1 0 0 1 0 0 0 0
0 0 0 0 0 1 0 0 1 0 0 1
0 0 1 0 1 0 0 1 0 0 0 1
0 0 0 1 0 0 0 0 0 0 1 0
0 0 1 0 0 0 0 0 1 1 0 0
1 0 0 1 0 0 0 0 0 1 0 0
1
2
3
4
5
6
7
8
9
10
11
12





Висновок: Під час виконання лабораторної роботи було опановано методи графічного представлення графів та принципи їх зберігання в пам'яті комп'ютера. Практично досліджено, як структурування даних у вигляді графів взаємодіє з системними ресурсами, що дозволило краще зрозуміти фундаментальні

принципи організації інформації в операційних системах. Набуті навички візуалізації та програмного представлення графів є базовими для подальшої розробки складних алгоритмічних систем.